

Spark vs. Ray: The Data Engineering Duel

DA3408 Assignment 3 M. Sai Teja (DA24B031) 5 October 2026

Code: <https://github.com/MADINENI-SAITEJA/AI-Ops-Assignment-3>

1 Summary

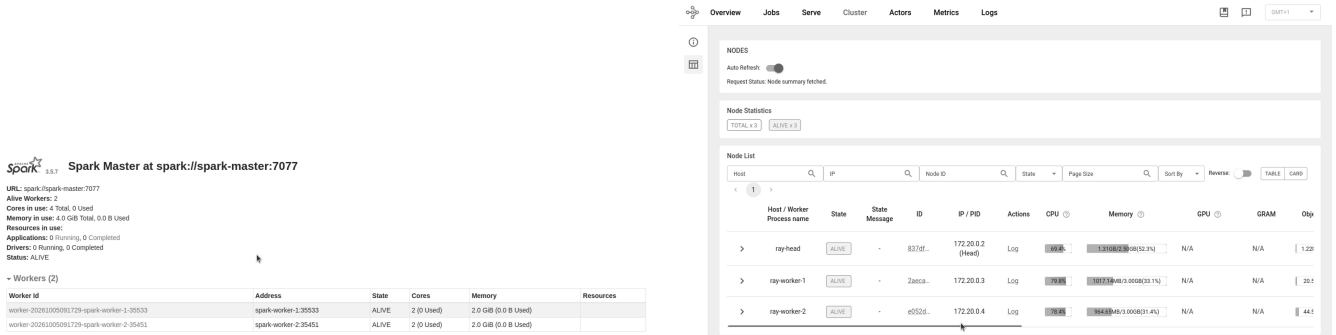
The same cleaning pipeline was implemented in Apache Spark and in Ray Data and run on a two-worker cluster for each framework, using the NYC yellow taxi trip records for January to June 2023 (19,493,620 input rows).

- Spark was faster for this pipeline on this hardware: 186.6 s and 253.2 s in two runs, against 780.7 s for Ray (one run). Ray was therefore $3.1\times$ to $4.2\times$ slower, and it also used more peak memory (6.67 vs. about 5.8 GiB for the cluster).
- The two outputs are identical: 18,253,326 rows, the same 30 columns and types, the same order-independent row hash, and no duplicate rows in either.
- Python UDF overhead (UDF time minus a baseline scan) was 50.2 s in Spark (one run) and 32.1 s in Ray (median of three runs, range 31.7 to 46.1 s), so Ray's was about $1.6\times$ lower. Spark's built-in expression was faster than Ray's vectorized one (3.7 s vs. 9.2 s median).
- Only six of the nine downloaded months were used, because Ray could not complete nine months inside the memory and disk of this VM (Section 7). All numbers come from my own runs, saved as JSON files in the repository, and the screenshots are in Figure 1 and the appendix.

2 Cluster setup

	Spark (standalone)	Ray (Ray Data)
Version	Spark 3.5.7, Python 3.11.17	Ray 2.49.2, Python 3.11.17, pyarrow 19.0.1
Topology	1 master, 2 workers (2 cores and 2 GiB executor memory each), 1 driver container (768 MB)	1 head node (no task CPUs), 2 workers (2 CPUs and a 1 GiB object store each)
Compute	4 executor cores, 4 GiB executor memory	4 CPUs, about 2.1 GiB total object store

Table 1: Cluster configuration.



(a) Spark Master UI (port 8080): 2 alive workers, 4 cores, 4.0 GiB. (b) Ray Dashboard (port 8265): 3 alive nodes (head and 2 workers), captured under load.

Figure 1: Cluster evidence for both frameworks.

The host is one 6-core, 7.2 GiB aarch64 Ubuntu VM (UTM/QEMU). Every node is a Docker container (3 GiB limit per worker) on one bridge network, and the two clusters ran one at a time because there was not enough RAM for both. The workers are therefore containers on one VM rather than separate machines, so network and disk are shared and the resource numbers describe a single 7.2 GiB host.

3 Pipeline and parity

`spark_clean.py` and `ray_clean.py` apply the same rules, which are recorded in the metrics file of every run. Timing covers reading through the finished Parquet export; cluster start-up and output inspection are excluded in both.

Step	Rule (identical in Spark and Ray)
Ingestion	Six monthly Parquet files and the taxi-zone lookup from a shared directory; schemas normalized to 19 source columns (the monthly files differ in physical types).
Cleansing	Drop rows with a null in any of the 19 columns; keep $1 \leq \text{passengers} \leq 8$, $0 < \text{distance} \leq 1000$, $0 < \text{duration} \leq 24 \text{ h}$, $\text{fares} \geq 0$, VendorID and $\text{RatecodeID} > 0$, store flag in {Y, N}; pickup within January to June 2023; global DISTINCT over all 19 columns across all files.
Transformation	Two INNER joins to the zone lookup (pickup and dropoff; blank labels become Unknown); one shared Python function computes <code>avg_speed_mph</code> , <code>fare_per_mile</code> and <code>fare_per_minute</code> (6 decimals), plus pickup hour and trip duration.
Export	Snappy Parquet, 64 files, 30 columns. Input 19,493,620 rows, output 18,253,326 rows (6.4% removed).

Table 2: Pipeline steps.

Check (6 months)	Spark	Ray	Result
Rows	18,253,326	18,253,326	match
Distinct rows (duplicates)	18,253,326 (0)	18,253,326 (0)	no duplicates
Columns and types	30	30	match
Row hash over all columns	same value for both		identical
1-month exact diff (EXCEPT ALL)	2,884,428 rows	2,884,428 rows	0 / 0 differences

Table 3: Output parity.

The six-month hash is an order-independent sum of per-row hashes over all columns, and both outputs gave the same value, 168357332003157618401821125 (Figure 7a). It is a hash comparison and not a row-by-row diff: a bucketed row-by-row verifier matched all 16 of the 128 buckets it completed (0 differences) before I stopped it for time. The one-month outputs were verified row by row (Figure 8b).

The rules recorded in the metrics files of the two runs compare equal, and so do the month range and the input and output row counts (Figure 2).

```
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/da3408_a3_repo$ cd ~/taxi_a3
python3 - <<'PY'
import json
s = json.load(open("artifacts/spark_6months_run1.json"))
r = json.load(open("artifacts/ray_joinfixed_6months_run1.json"))
print("Spark rules == Ray rules:", s["rules"] == r["rules"])
print("Same months:", s["months"] == r["months"], "| same input rows:", s["input_rows"] == r["input_rows"])
print("Output rows:", s["output"]["rows"], r["output"]["rows"])
PY
Spark rules == Ray rules: True
Same months: True | same input rows: True
Output rows: 18253326 18253326
```

Figure 2: Terminal: the `rules` recorded in the Spark and Ray run files are equal; same months, same input rows and the same 18,253,326 output rows.

4 Performance (January to June 2023)

Metric	Spark run 1	Spark run 2	Ray	Ray / Spark
Execution time, ingestion to export	186.6 s	253.2 s	780.7 s	4.2 / 3.1
Peak CPU, cluster (400% = 4 cores)	487%	437%	503%	similar
Peak memory, cluster (working set)	5.83 GiB	5.81 GiB	6.67 GiB	1.14 – 1.15
Output on disk (64 files)	695 MB	not read back	937 MB	1.35

Table 4: Measured results, one run per column.

Run 2 of Spark was read back for its headline values only, so its output size is missing; parity was verified on run 1’s output, and run 2 wrote the same 18,253,326 rows. Spark’s two runs differ by 36%, so the gap to Ray is a range and not a single number. Memory is the working set, since the separately recorded “charged” memory was about 6.9 GiB for both and seems to include page cache. Evidence: Figures 7b and 6b–8b. Ray’s output is larger on disk (937 vs. 695 MB), but the row hash shows the data is identical, so this is a Parquet encoding difference.

Why Ray was slower here

Both frameworks kept the four cores close to saturated, so the gap is not idle CPU. The run logs point to Ray’s data movement under a small memory budget (this is a diagnosis from the logs, not a profile):

- **Super-linear scaling.** Ray ran January alone (2.88M output rows) in 24.6 s, but six months (18.25M rows, 6.3 times more) in 778.6 s, which is 31.6 times the time.

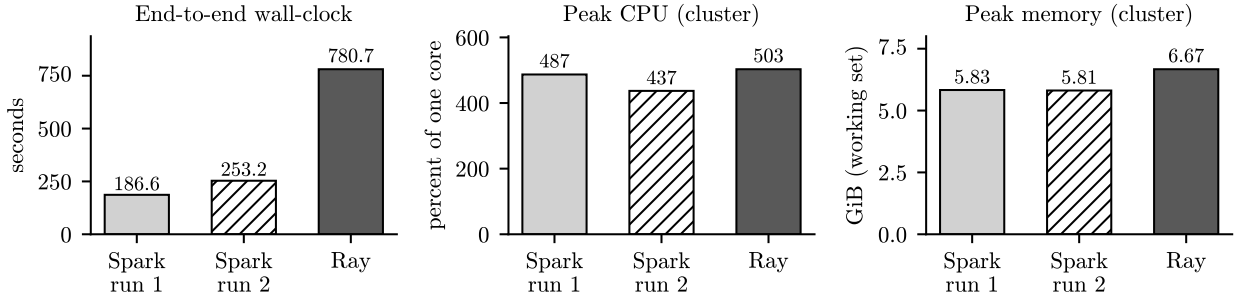


Figure 3: End-to-end time and cluster peaks, sampled from container metrics by the measurement wrapper.

- **Global DISTINCT is a shuffle.** Ray Data has no hash distinct, so the implementation gives each row a hash bucket of its 19 source fields, range-partitions all rows by that bucket with a Ray sort (an all-to-all barrier), and then runs an exact Arrow DISTINCT inside each bucket. The shuffled data (2.7 GB or more) is larger than the 2.1 GiB combined object store, whereas Spark’s distinct is a hash aggregation with managed spill.
- **The failures agree.** The nine-month Ray attempts ended in an out-of-memory kill and “local disk is full”, and an earlier six-month variant lost objects and stalled (Section 7).

5 Python UDF deep-dive

To isolate the UDF, I read the finished output (18.25M rows) in each framework and ran the same per-row Python function (speed = distance / hours, from the pickup and dropoff timestamps) three ways: a baseline scan of the three columns, the Python UDF, and a native version. A warm-up read comes before timing. Spark writes to a no-op sink, and Ray materializes its blocks.

Measurement (seconds)	Spark	Ray run 1	Ray run 2	Ray run 3	Ray median
Baseline scan (3 columns)	0.98	2.74	2.84	2.88	2.84
Python UDF, total	51.14	34.46	34.90	48.95	34.90
Native / vectorized, total	3.73	5.68	9.15	10.26	9.15
UDF overhead (UDF total – baseline)	50.15	31.72	32.06	46.07	32.06
Native overhead	2.75	2.94	6.31	7.38	6.31
UDF total / native total	13.7	6.1	3.8	4.8	4.8

Table 5: UDF micro-benchmark on the six-month output. Spark is a single run; Ray was run three times.

The raw values are in Figure 4.

```
saiteja-ubuntu-31@saiteja-ubuntu-31-QEMU-Virtual-Machine:~/taxi_a3$ python3 - <<'PY'
import json
for f in ["udf_spark_run1", "udf_ray_run1", "udf_ray_run2", "udf_ray_run3"]:
    r = json.load(open(f"artifacts/{f}.json"))
    print(f, {k: round(v, 2) for k, v in r.items() if k != "framework"})
PY
udf_spark_run1 {'base_s': 0.98, 'udf_total_s': 51.14, 'native_total_s': 3.73, 'udf_overhead_s': 50.15, 'native_overhead_s': 2.75}
udf_ray_run1 {'base_s': 2.74, 'udf_total_s': 34.46, 'native_total_s': 5.68, 'udf_overhead_s': 31.72, 'native_overhead_s': 2.94}
udf_ray_run2 {'base_s': 2.84, 'udf_total_s': 34.9, 'native_total_s': 9.15, 'udf_overhead_s': 32.06, 'native_overhead_s': 6.31}
udf_ray_run3 {'base_s': 2.88, 'udf_total_s': 48.95, 'native_total_s': 10.26, 'udf_overhead_s': 46.07, 'native_overhead_s': 7.38}
```

Figure 4: Terminal: the saved UDF benchmark files for Spark (run 1) and Ray (runs 1 to 3).

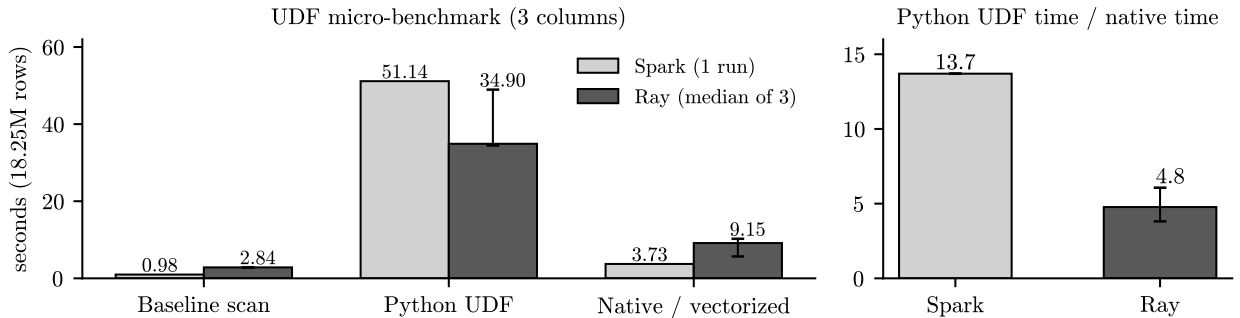


Figure 5: UDF micro-benchmark. Bars: Spark one run, Ray median of three; whiskers show the Ray minimum and maximum. Evidence: Figures 4, 6a and 7a.

Spark’s Python UDF is 13.7 times slower than its own native expression, because every row crosses the JVM–Python boundary and is serialized in both directions. Ray’s batches already live in Python, so the penalty is smaller

(3.8 to 6.1 times, median 4.8), and its median UDF overhead is 18.1 s lower (32.1 vs. 50.2 s, about 1.6 $\times$). That is the Python-native advantage. Two limits apply. Ray’s overhead varied between runs (31.7 to 46.1 s) and its slowest run is close to Spark’s single run, so the advantage is indicative and not precise. It is also small: 18.1 s is about 3% of the 527 to 594 s end-to-end gap, so the UDF does not explain why Ray lost; the relational stages do. Without a Python UDF, Ray was not ahead: its vectorized path (median 9.15 s) was slower than Spark’s native expression (3.73 s).

Caveats: Spark’s UDF figure is one run, and Ray’s three runs spread widely (run 3 was about 40% slower than the median, probably noise on the shared VM). Spark passes `datetime` objects and Ray passes pandas `Timestamp` objects to the same function, and Spark’s native expression truncates to whole seconds. The baselines use different sinks, so subtracting them is only approximate. An earlier Ray run without a warm-up (baseline 7.07 s, UDF 29.99 s) is excluded because its cold-cache baseline made the native overhead negative.

6 Conclusion: which framework for which project

	Spark (BI-first)	Ray (AI-first)
Workload	Relational ETL: joins, global distinct, aggregations, SQL, scheduled reporting	Python-heavy feature engineering, ML preprocessing that feeds training, GPU or actor pipelines
Evidence here	3.1 $\times$ to 4.2 $\times$ faster end to end, lower memory, no tuning crisis; native expressions are cheapest	Python UDF overhead about 1.6 $\times$ lower (median); data stays in pandas/Arrow in the same runtime as the model code
Cost	Python UDFs are slow (13.7 $\times$ native); avoid them or use vectorized pandas UDFs	Relational operators (distinct, shuffles) need hand tuning and a lot of object-store memory

Table 6: Choosing between the two.

Verdict. For this preprocessing pipeline on this hardware, Spark wins. For a new AI-first project I would still pick Ray for the Python-native, model-facing stages, but let Spark (or another hash-based engine) do the heavy relational cleaning, and give Ray a larger object store than the 2.1 GiB available here.

7 Performance tuning note

LLM assistants (ChatGPT and Claude) helped translate the pipeline between PySpark and Ray Data, suggested the Ray tuning below, and wrote the benchmark scripts. I set up and configured both clusters, ran every benchmark and took every screenshot myself, and no performance number was generated by an AI. Each AI suggestion was tested on the VM:

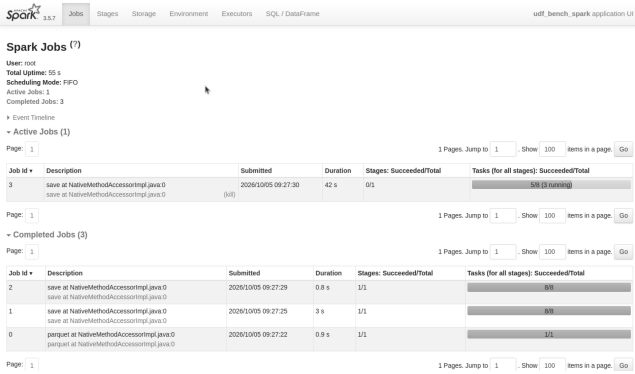
AI-suggested Ray change	Real result on this VM
Sort-based global distinct, 256 buckets, one read operator for all files	Kept. Correct (0 duplicates, hash parity).
Pull-based sort shuffle instead of the default push-based shuffle	1 month: 20.1 s vs. 62.0 s. Kept.
Nine months, one read operator per file plus union	No result; stopped after about 20 min.
Nine months, single bounded-batch reader	Container out-of-memory kill (exit 137).
Nine months, 64 MiB blocks	<code>OutOfDiskError</code> during the sort reduce.
Six months, two separate Ray joins to the zone table	Lost-object recovery, then failure after about 16 min.
Six months, one native join on a combined pickup/dropoff lookup (70,225 pairs, equivalent to both joins)	Succeeded in 780.7 s with exact parity. Used for the reported number.

Table 7: Ray tuning history on this VM.

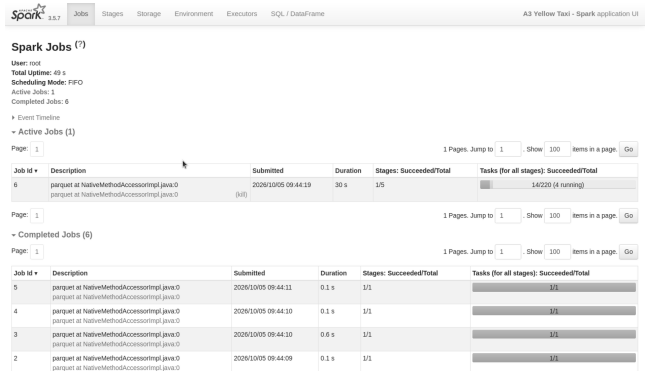
Limits. One VM (6 cores, 7.2 GiB) hosts every node; six months, not nine; two Spark end-to-end runs (186.6 s and 253.2 s) and one Ray run; the UDF benchmark has three Ray runs and one Spark run; the six-month parity check is a row hash. Absolute times would differ on real multi-machine clusters.

Appendix: evidence screenshots

The screenshots are also submitted as separate image files; they are converted to grayscale here.

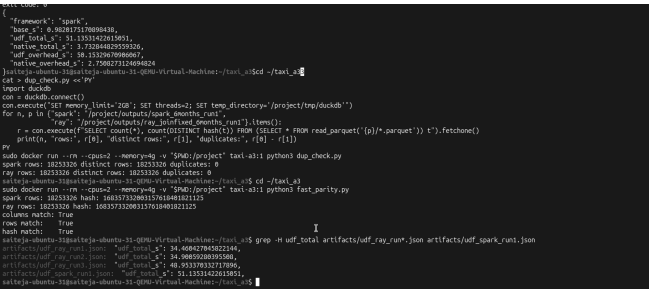


(a) UDF benchmark: three jobs done, the Python-UDF job running.

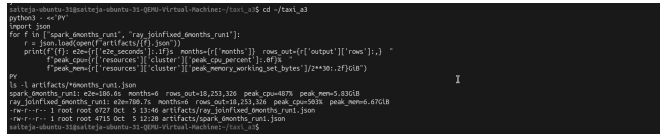


(b) Main pipeline (run 2): job 6 at 14/220 tasks, 4 running.

Figure 6: Spark UI (port 4040), application “A3 Yellow Taxi - Spark” on the right. Four running tasks match the 4 executor cores.

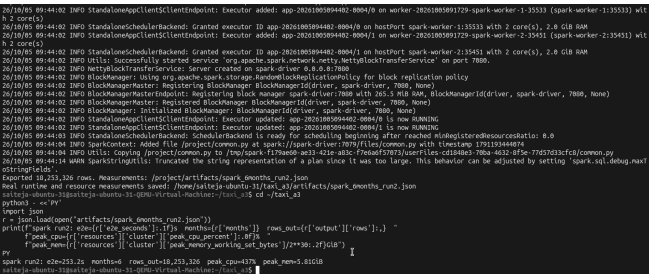


(a) Spark UDF benchmark result (exit code 0), duplicate check (0 duplicates in both outputs), row-hash parity (all True), and the four UDF totals.



(b) Headline results read from the saved run files: Spark 186.6 s and Ray 780.7 s, both 6 months and 18,253,326 output rows.

Figure 7: Terminal screenshots: parity, duplicates and headline results.



(a) Spark run 2: executors on both workers (2 cores, 2.0 GiB each), 18,253,326 rows exported, 253.2 s, 437% peak CPU, 5.81 GiB peak memory. The run command is screenshot 8 in the separate image files.



(b) Ray side: one-month exact output parity (differences 0/0) and the six-month Ray run (command, 778.55 s Ray Data execution, 18,253,326 rows exported).

Figure 8: Terminal screenshots: Spark run 2 and the Ray run.